Daws86s-Interview Preparation Series(DevSecOps-Day1)

SDLC (Requirements → Design → Code → Test → Deploy → Maintain) Waterfall and agile comes under this linear procedure

DevOps focuses on what users never see —
so users never feel it.

Devops is the process of building or deploying and testing the applications continuously. Basically if we deploy and test the code on the same day we can get maximum feedback cycles to improve our product before delivering it to the end user. We use multiple tools to make the process smooth. Continuous development integration, deployment, delivery, testing, monitoring, application security etc are always a part of DevOps practices. Code and infra should be always same across environments. Automation should be our primary resposibility.

Onprem vs Cloud

On-premise infrastructure is when a company runs everything inside its own data center. The servers, storage, networking devices — everything is physically owned by the organization. From a DevOps point of view, this means we're responsible for almost everything: buying hardware, installing operating systems, configuring networks, handling failures, patching servers, and planning capacity well in advance. If traffic suddenly increases, we can't scale instantly — we need to procure new servers, which can take weeks or even months. On-prem works well for legacy systems or organizations with strict compliance requirements, but it comes with high upfront cost and a lot of operational effort.

Cloud infrastructure, on the other hand, removes the hardware headache completely. We don't own servers — we rent computing power from cloud providers and pay only for what we use. For DevOps teams, this is a huge shift. Infrastructure can be created using code, scaled automatically, and destroyed when not needed. If traffic spikes, new instances come up in minutes through auto-scaling. Most operational tasks like hardware failures, data center maintenance, and basic availability are handled by the cloud provider, allowing DevOps engineers to focus more on CI/CD, reliability, monitoring, and performance rather than firefighting hardware issues.

The biggest practical difference between on-prem and cloud is **speed and flexibility**. In on-prem, planning mistakes are expensive — if you under-provision, applications crash; if you over-provision, money is wasted. In the cloud, you can start small, observe usage, and scale as required. This aligns perfectly with DevOps principles like continuous delivery and rapid iteration. Infrastructure becomes part of the deployment pipeline rather than a separate manual process.

From a **cost perspective**, on-prem requires heavy capital investment upfront, while cloud follows an operational cost model. This means cloud makes experimentation easier — teams can spin up environments for testing, staging, or demos and tear them down after use. In on-prem setups, teams often avoid creating new environments because of limited resources.

In terms of **reliability and disaster recovery**, on-prem setups usually need a secondary data center, which is expensive and complex. In the cloud, high availability and disaster recovery are much easier to implement using multiple availability zones and regions. Backup, replication, and failover are either built-in or much simpler to automate.

From a **DevOps mindset**, on-prem DevOps engineers spend more time managing infrastructure and resolving operational issues. Cloud DevOps engineers spend more time improving automation, deployment pipelines, monitoring, scalability, and system resilience. That's why most modern DevOps practices naturally fit better with cloud environments

PreRequisites

Operating System and Kernel

An operating system is basically the middle layer that makes sure applications and hardware can work together smoothly without stepping on each other. As a DevOps engineer, you can think of the OS as the manager of the system—it decides how CPU time is shared, how memory is allocated, how files are read or written, and who is allowed to do what. At the core of the OS sits the kernel, which is the most critical part because it directly talks to the hardware. The kernel handles things like process scheduling, memory management, disk and network operations, and system security. Applications run in user space with limited permissions and must request the kernel's help through system calls whenever they need hardware access. This separation is what keeps systems stable and secure. In real-world DevOps work, containers, servers, and cloud VMs all rely heavily on the Linux kernel, which is why understanding how the OS and kernel work together helps you troubleshoot performance issues, crashes, and security

AMI / Machine Image (Golden Image Concept)

In real-world cloud environments, images are created using automation, not manual setup. DevOps teams start with a base OS image provided by the cloud platform and use scripts or configuration tools to install required packages, apply security updates, and add standard configurations. This process runs on a temporary instance, which is validated and then converted into a reusable image. The temporary instance is discarded, and the image becomes the standard template for launching new servers.

These images are versioned and rebuilt whenever there are OS patches, security fixes, or dependency changes. In production, images are used with auto scaling and deployments so that new instances are launched from the same tested image instead of fixing servers manually. This approach ensures consistency, faster recovery, and predictable behavior across environments.

Capacity

In real projects, capacity selection does not start with randomly choosing instance types like t3.micro or t3.large. It starts with understanding the workload. DevOps teams first identify what the application does, how many users it serves, and whether it is CPU-intensive, memory-intensive, or mostly I/O-based. Lightweight services, background jobs, or low-traffic environments usually begin with smaller instances such as t3.micro or t3.small, while heavier applications require larger sizes.

In production, capacity is almost never fixed permanently. Teams usually start with a small, cost-effective instance and observe real usage through monitoring. Metrics like CPU utilization, memory usage, response time, and error rate are tracked. If the application consistently runs near its limits, the instance size is increased or additional instances are added. This data-driven approach avoids over-provisioning and ensures performance requirements are met.

Burstable instance types like t3 are commonly used for applications with variable or unpredictable traffic. These instances can handle short spikes in CPU usage while remaining cost-efficient during normal load. For steady or high-performance workloads, teams switch to non-burstable or compute-optimized instances. The choice is always guided by application behavior rather than guesswork.

In real DevOps setups, capacity decisions are combined with auto scaling. Instead of choosing one large instance, teams often use multiple smaller instances behind a load balancer. Scaling rules are defined so new instances are added when CPU or traffic increases and removed when demand drops. This provides better availability, fault tolerance, and cost control than relying on a single large server.

Authentication

In real projects, authentication starts with identifying who or what needs access. Human users usually authenticate through a centralized identity provider using single sign-on, while applications authenticate using roles rather than passwords. For example, when an application running on a virtual machine needs to read from object storage or connect to a database, it is assigned a role that allows that action. No credentials are stored in the application code or configuration files. This approach is used consistently across environments because it reduces security risks and simplifies credential management.

Authorization

Authorization is then used to control exactly what actions an authenticated identity can perform. In production environments, permissions are never broad or generic. Each user or service is given only the permissions required to perform its task. For instance, a CI/CD pipeline might be allowed to deploy application code but not modify networking resources. Authorization rules are reviewed regularly, version-controlled, and updated carefully to avoid accidental privilege escalation. This is especially important in large teams where multiple services and engineers share the same environment.

Firewalls and Security Groups

Firewalls are used in real cloud environments to define the first level of network protection. Teams typically allow only essential inbound traffic such as HTTPS from the internet and block all unnecessary ports. Outbound traffic may also be restricted for security and compliance reasons. Firewall rules are adjusted carefully and tested, because an incorrect rule can instantly make an application unreachable. Since cloud firewalls are software-defined, updates can be applied quickly, which is useful during incident response.

Security groups provide more precise, resource-level control and are heavily used in production. In a typical setup, a load balancer has a security group that allows traffic from the internet, application servers have a security group that allows traffic only from the load balancer, and databases have a security group that allows traffic only from application servers. This layered approach ensures that even if one component is exposed, the rest of the system remains protected. Security groups are stateful, which simplifies return traffic handling.

Secrets Management

Secrets management is treated as a critical responsibility in real-world DevOps. Database credentials, API keys, and tokens are stored securely and injected into applications at runtime. Secrets are rotated periodically and never committed to source control or baked into images. During incidents or audits, DevOps teams can rotate secrets without rebuilding applications, which greatly improves security posture.

Networking

Networking fundamentals are applied constantly in real environments. DevOps engineers design private and public networks, manage routing rules, configure DNS, and ensure services can communicate securely. When an application is not reachable, the first checks usually involve network paths, firewall rules, and security groups rather than application code. Strong networking knowledge significantly reduces troubleshooting time.

Monitoring and Logging

Monitoring and logging are always enabled in production systems. Teams track metrics like CPU usage, memory consumption, request latency, and error rates to understand system health. Logs are collected centrally to help diagnose issues and analyze incidents. Alerts are configured so DevOps engineers are notified before problems affect users. Without observability, reliable operations are impossible.

Real World Readiness

environment separation. In real projects, development, testing, staging, and production are never mixed. Each environment has its own resources, access rules, and sometimes even separate accounts or subscriptions. This prevents testing activity from affecting real users and allows controlled releases. DevOps engineers must understand why environments are isolated and how configuration differs across them.

configuration management. In production systems, application behavior is never hardcoded. Configuration such as database endpoints, feature flags, and environment-specific values are injected at runtime. This allows the same image or application build to run in multiple environments without modification. Mismanaging configuration is a common cause of outages.

Immutability is another real-world principle DevOps engineers follow. Once a server or instance is created, it is not modified manually. Any change requires creating a new image or deployment and replacing the old one. This reduces unpredictable behavior and makes rollback easier during failures.

Understanding **failure as a normal event** is also critical. In cloud systems, components are expected to fail. DevOps engineers design systems assuming instances can disappear at any time. Health checks, auto recovery, and redundancy are used so failures don’t impact users. This mindset separates real DevOps engineers from traditional system administrators.

Important area is **change management**. In real environments, every change is tracked, reviewed, and reversible. Deployments are automated, changes are logged, and rollback strategies are planned in advance. DevOps engineers focus on reducing risk while increasing deployment frequency.

Basic understanding of **cost awareness** is also expected. DevOps engineers monitor usage, identify unused resources, and design systems that scale efficiently. Cost optimization is not a finance task alone — it’s a technical responsibility as well.

DevOps engineers must understand **observability readiness** before systems go live. This means ensuring metrics, logs, and alerts exist before incidents happen. Many failures become severe simply because visibility was missing.

Pre Implementation strategies

Source of Truth & Version Control
In real DevOps environments, everything that defines how a system works is stored in version control and treated as the single source of truth. Application code, infrastructure definitions, configuration files, access policies, and deployment logic are all tracked and versioned. Nothing critical should exist only on a server or in someone's memory. This ensures changes are traceable, reversible, and reviewable. When something breaks, DevOps engineers rely on version history to understand what changed and roll back safely. Without a clear source of truth, automation and reliability quickly fall apart.

Artifacts & Build Outputs
Another important concept is understanding what an artifact is and why it matters. In real projects, code is built once and produces an artifact, such as a binary, package, or container image. That same artifact is promoted across environments from development to testing to production. Rebuilding the application separately in each environment introduces inconsistency and risk. DevOps practices ensure that what is tested is exactly what gets deployed, which improves reliability and simplifies debugging when issues occur in production.

Deployment Strategies
Deployment is never just about pushing code; it is about how that code is released safely. In real environments, DevOps engineers use controlled deployment strategies to reduce risk. Instead of replacing everything at once, systems are updated gradually or in parallel so that failures can be detected early. If something goes wrong, traffic can be shifted back quickly. This approach allows teams to deploy frequently while minimizing user impact.

Stateless vs Stateful Components
Understanding the difference between stateless and stateful components is critical before designing scalable systems. Stateless services do not store user or session data locally and can be scaled horizontally without complex coordination. Stateful components, such as databases or message queues, require careful handling because data consistency and persistence matter. In real projects, DevOps engineers design systems so that most application logic is stateless, while stateful components are isolated and protected. This separation makes scaling, recovery, and automation much simpler.

Backup vs Disaster Recovery
Backup and disaster recovery are related but not the same, and this distinction is important in production systems. Backups protect data from accidental deletion or corruption, while disaster recovery ensures that the entire system can be restored and continue operating after a major failure. In real environments, DevOps engineers define recovery time and recovery point expectations and design systems accordingly. Simply having backups is not enough if the application cannot be restored within acceptable time limits.

Access Lifecycle Management
Access management in real organizations follows a lifecycle. When someone joins a team, they are granted only the access required for their role. When responsibilities change, permissions are adjusted. When someone leaves, access is revoked immediately. DevOps engineers must design authentication and authorization systems that support this lifecycle cleanly and securely. Failing to manage access properly is a common cause of security incidents in production systems.

Shared Responsibility Model
Cloud environments operate on a shared responsibility model, which DevOps engineers must clearly understand. The cloud provider is responsible for the underlying infrastructure, physical security, and core platform availability, while the DevOps team is responsible for operating systems, applications, configurations, access controls, and data protection. Many real-world incidents happen when teams assume the provider handles responsibilities that actually belong to them. Understanding this boundary is essential for secure and reliable cloud operations.

Reliability Targets & Error Budget Mindset
Modern DevOps teams operate with clear reliability expectations. Systems are designed with agreed-upon availability and performance targets, and teams understand that some level of failure is acceptable. Instead of aiming for zero failures, DevOps engineers balance delivery speed with system stability. When incidents occur, teams learn from them and improve the system. This mindset ensures sustainable operations rather than fragile perfection.